

Parte 3. Interpretación de comandos

1. Analiza

Planteamiento:

git status

git add README.md

git commit -m "Actualiza documentación"

git push

Explica qué ocurre en cada instrucción.

Respuesta y explicación:

- **`git status`** - Muestra el estado actual del repositorio de trabajo: en qué rama estoy, qué archivos han sido modificados pero no están en el área de preparación (unstaged), qué archivos ya están listos para el próximo commit (staged) y qué archivos nuevos no están siendo rastreados todavía (untracked). No modifica nada, solo informa.

- **`git add README.md`** - Toma los cambios realizados en el archivo `README.md` y los coloca en el **área de preparación** (staging area / índice). Esto significa que ese archivo queda marcado para ser incluido en el próximo commit, pero el cambio todavía no ha sido registrado de forma permanente en el historial.

- **`git commit -m "Actualiza documentación"`** - Toma todo lo que está en el área de preparación (en este caso, el `README.md` modificado) y crea una nueva instantánea (commit) en el **historial local** del repositorio, junto con el mensaje `"Actualiza documentación"` que describe el cambio realizado. Esta operación es completamente local; todavía no afecta a GitHub.

- **`git push`** - Envía los commits que existen en mi repositorio local (incluyendo el que acabo de crear) hacia el repositorio remoto configurado (por ejemplo, `origin`` en GitHub), actualizando la rama remota para que refleje el mismo historial que tengo localmente. Es en este paso cuando el cambio se vuelve visible para otras personas en GitHub.

2. Identifica qué falta

Caso A

Planteamiento:

Modificar archivo

|

git add .

|

¿?

|

git push

Indica qué operación falta y explica su función.

Respuesta: Falta `git commit -m "mensaje descriptivo"`.

Explicación: `git add .` solamente mueve todos los cambios detectados al área de preparación; todavía no existe ningún registro permanente de esos cambios en el historial del repositorio. Antes de poder subir (push) algo al repositorio remoto, es necesario empaquetar esos cambios preparados en un commit, que es la unidad mínima de historial que Git puede transferir entre repositorios. Sin el commit, no hay nada nuevo que `git push` pueda enviar.

Caso B

Planteamiento:

Repositorio GitHub

|

¿?

|

Repositorio local

Indica qué operación utilizarías y explica por qué.

Respuesta: Utilizaría `git clone <url-del-repositorio>`.

Explicación: Este esquema representa el momento en que **todavía no existe** una copia local del repositorio, sino que se va a crear por primera vez a partir de lo que existe en GitHub. `git clone` descarga el repositorio completo (todo su historial de commits, ramas y archivos) desde GitHub y lo coloca en una carpeta nueva en mi computadora, dejando automáticamente configurado el remoto `origin` apuntando hacia ese repositorio de GitHub.

Caso C

Planteamiento:

Repositorio remoto actualizado

|

¿?

|

Repositorio local actualizado

Indica qué operación utilizarías y explica por qué.

Respuesta: Utilizaría `git pull`.

Explicación: A diferencia del Caso B, aquí ya existe una copia local previa del repositorio (por eso el resultado es "repositorio local **actualizado**" y no un repositorio local recién creado). El repositorio remoto tiene commits nuevos que mi copia local todavía no tiene. `git pull` descarga esos commits nuevos desde el remoto (`git fetch`) y los integra automáticamente en mi rama local actual (`git merge`), dejando mi repositorio local sincronizado con el estado más reciente del repositorio remoto.